



Report: Mankala Game

Course Code: CSE 403

Course Title: Artificial Intelligence and Expert Systems

Submitted to:

Md. Shafayatul Haque

Lecturer

Department of CSE

Submitted by:

Name: Rabia Sultana Aunti

ID: 20201019

1. Problem Title

Two-Player Console Mankala Game in Python

2. Problem Description

Game Mechanics: Mancala is a two-player turn-based board game with 12 pits and 2 storages where stones move counter-clockwise.

1. **Objective:** The core goal is to maximize the final stone count in one's own storage bin.
2. **Framework:** The game is modeled as an adversarial search problem using a maximizer and a minimizer player.
3. **AI Implementation:** An intelligent AI agent uses the Minimax Algorithm to simulate future states and choose moves.
4. **Optimization:** Alpha-Beta Pruning is integrated to remove redundant branches, saving computation time and memory overhead.
5. **Heuristics & UI:** The system employs four distinct heuristics for state evaluation within a premium web-based Graphical User Interface.

3. Tools and Languages Used

Programming Language: Python 3

Libraries & Modules:

copy – Deep-copies board states during tree simulations.

random – Resolves tie-breaking scenarios between identical heuristic scores.

time – Adds realistic gameplay pacing delays during AI turns.

ipywidgets – Builds the interactive layout for pits and stores.

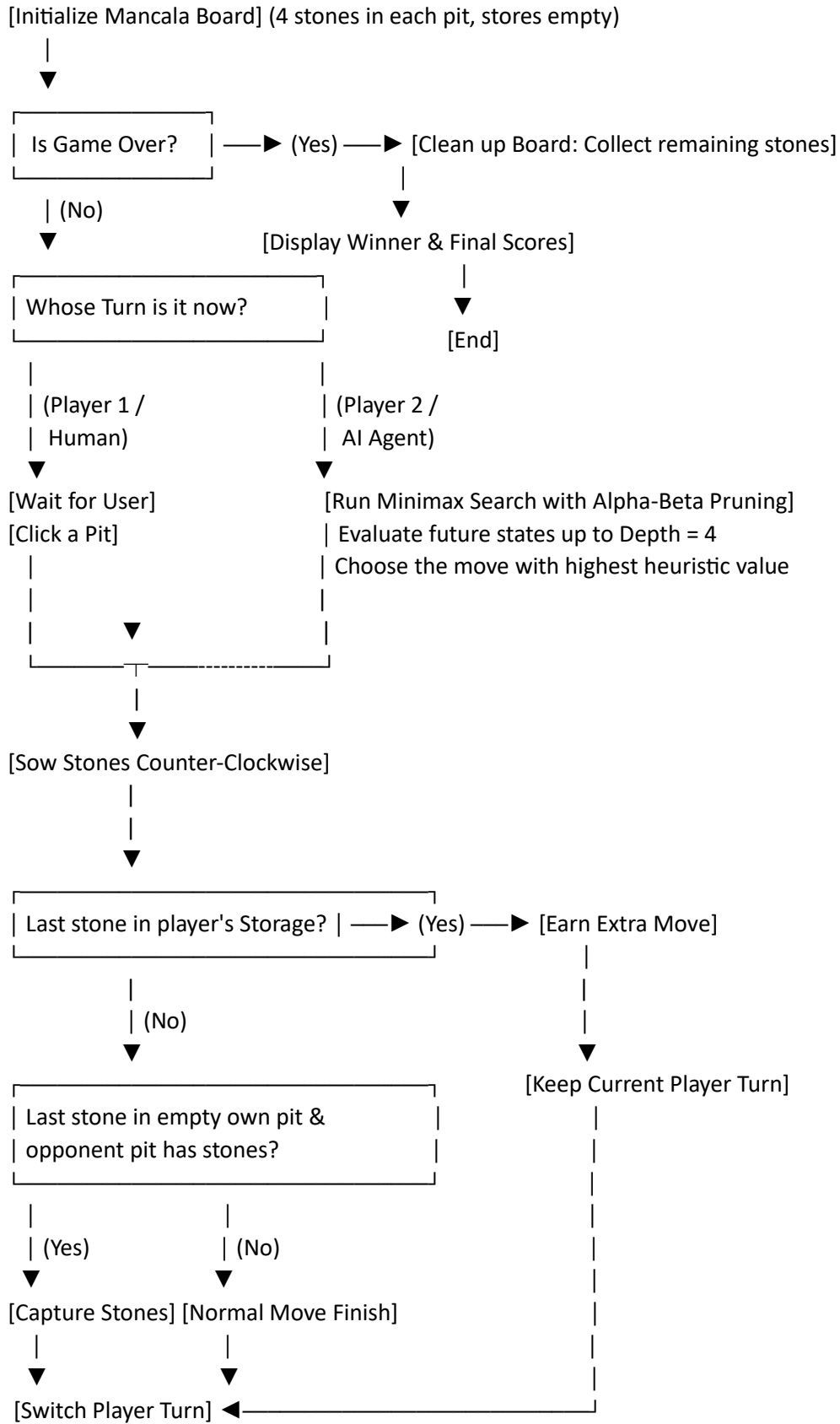
IPython.display – Injects custom CSS for wood-theme and board rendering.

Platform & IDE: Google Colab (Cloud-based web interface).

4. Game Flowchart

[Start Game]







(Loop back to "Is Game Over?")

5. Sample Input/Output (Gameplay Snapshot)

The simulation was implemented in Python.

Source Code:

```
[ ] 1 import ipywidgets as widgets
    2 from IPython.display import display, clear_output

[ ] 1 import copy
    2 import random
    3 import time
    4 import ipywidgets as widgets
    5 from IPython.display import display, HTML
    6
    7 #Board Indexes and Configuration
    8
    9 P1_PITS = list(range(0, 6))
   10 P1_STORE = 6
   11 P2_PITS = list(range(7, 13))
   12 P2_STORE = 13
   13
   14 class MancalaBoard:
   15     def __init__(self):
   16         self.board = [4, 4, 4, 4, 4, 4, 0, 4, 4, 4, 4, 4, 4, 0]
   17         self.current_player = 1
   18         self.extra_move_earned = False
   19         self.stones_captured = 0
   20
   21     def is_game_over(self):
   22         p1_empty = all(self.board[i] == 0 for i in P1_PITS)
   23         p2_empty = all(self.board[i] == 0 for i in P2_PITS)
   24         return p1_empty or p2_empty
   25
   26     def clean_up_board(self):
   27         p1_rem = sum(self.board[i] for i in P1_PITS)
   28         p2_rem = sum(self.board[i] for i in P2_PITS)
   29         self.board[P1_STORE] += p1_rem
   30         self.board[P2_STORE] += p2_rem
   31         for i in P1_PITS + P2_PITS:
   32             self.board[i] = 0
   33
   34     def get_valid_moves(self, player):
   35         pits = P1_PITS if player == 1 else P2_PITS
   36         return [i for i in pits if self.board[i] > 0]
```

```

34     def get_valid_moves(self, player):
35         pits = P1_PITS if player == 1 else P2_PITS
36         return [i for i in pits if self.board[i] > 0]
37
38     def make_move(self, pit_index):
39         stones = self.board[pit_index]
40         self.board[pit_index] = 0
41         current_idx = pit_index
42         player = self.current_player
43         self.extra_move_earned = False
44         self.stones_captured = 0
45
46         while stones > 0:
47             current_idx = (current_idx + 1) % 14
48             if player == 1 and current_idx == P2_STORE:
49                 continue
50             if player == 2 and current_idx == P1_STORE:
51                 continue
52             self.board[current_idx] += 1
53             stones -= 1
54
55             if player == 1 and current_idx == P1_STORE:
56                 self.extra_move_earned = True
57             elif player == 2 and current_idx == P2_STORE:
58                 self.extra_move_earned = True
59
60         my_pits = P1_PITS if player == 1 else P2_PITS
61         my_store = P1_STORE if player == 1 else P2_STORE
62
63         if current_idx in my_pits and self.board[current_idx] == 1:
64             opposite_idx = 12 - current_idx
65             if self.board[opposite_idx] > 0:
66                 captured = self.board[current_idx] + self.board[opposite_idx]
67                 self.board[my_store] += captured
68                 self.stones_captured = captured
69                 self.board[current_idx] = 0
70                 self.board[opposite_idx] = 0
71
72         if not self.extra_move_earned:
73             self.current_player = 2 if player == 1 else 1
74

```

```

74         if self.is_game_over():
75             self.clean_up_board()
76
77     #AI Logic (Minimax Alpha-Beta)
78     def evaluate_heuristic(board_state, player, heuristic_type, W1=2, W2=1, W3=3, W4=4):
79         my_store = board_state.board[P1_STORE] if player == 1 else board_state.board[P2_STORE]
80         opp_store = board_state.board[P2_STORE] if player == 1 else board_state.board[P1_STORE]
81         my_side_stones = sum(board_state.board[i] for i in (P1_PITS if player == 1 else P2_PITS))
82         opp_side_stones = sum(board_state.board[i] for i in (P2_PITS if player == 1 else P1_PITS))
83         h1 = my_store - opp_store
84         if heuristic_type == 1: return h1
85         h2 = W1 * h1 + W2 * (my_side_stones - opp_side_stones)
86         if heuristic_type == 2: return h2
87         add_move = 1 if board_state.extra_move_earned else 0
88         h3 = h2 + W3 * add_move
89         if heuristic_type == 3: return h3
90         return h3 + W4 * board_state.stones_captured
91
92     def minimax_alpha_beta(board_state, depth, alpha, beta, maximizing_player, player, heuristic_type):
93         if depth == 0 or board_state.is_game_over():
94             return evaluate_heuristic(board_state, player, heuristic_type), None
95         valid_moves = board_state.get_valid_moves(board_state.current_player)
96         if not valid_moves:
97             next_state = copy.deepcopy(board_state)
98             next_state.current_player = 2 if next_state.current_player == 1 else 1
99             return minimax_alpha_beta(next_state, depth - 1, alpha, beta, not maximizing_player, player, heuristic_type)[0], None
100         best_move = random.choice(valid_moves) if valid_moves else None
101         if maximizing_player:
102             max_eval = float('-inf')
103             for move in valid_moves:
104                 next_state = copy.deepcopy(board_state)
105                 next_state.make_move(move)
106                 is_max = maximizing_player if next_state.current_player == board_state.current_player else not maximizing_player
107                 evaluation, _ = minimax_alpha_beta(next_state, depth - 1, alpha, beta, is_max, player, heuristic_type)
108                 if evaluation > max_eval:
109                     max_eval = evaluation
110                     best_move = move
111                 alpha = max(alpha, evaluation)
112                 if beta <= alpha: break
113             return max_eval, best_move
114

```

```

114         return max_eval, best_move
115     else:
116         min_eval = float('inf')
117         for move in valid_moves:
118             next_state = copy.deepcopy(board_state)
119             next_state.make_move(move)
120             is_max = maximizing_player if next_state.current_player == board_state.current_player else not maximizing_player
121             evaluation, _ = minimax_alpha_beta(next_state, depth - 1, alpha, beta, is_max, player, heuristic_type)
122             if evaluation < min_eval:
123                 min_eval = evaluation
124                 best_move = move
125             beta = min(beta, evaluation)
126             if beta <= alpha: break
127         return min_eval, best_move
128
129
130 # PREMIUM LIGHT WOODEN STYLE CSS WITH ENHANCED STATUS ROW
131
132 custom_css = """
133 <style>
134     .mancala-board {
135         background-color: #F5DEB3 !important;
136         background-image: linear-gradient(145deg, #F5DEB3, #E6C280) !important;
137         border: 6px solid #8B4513 !important;
138         border-radius: 25px !important;
139         padding: 25px !important;
140         box-shadow: 0px 12px 24px rgba(139, 69, 19, 0.3) !important;
141     }
142
143     .mancala-pit, .mancala-pit.lm-Widget {
144         background-color: #D7CCC8 !important;
145         background-image: none !important;
146         border: 3px solid #5D4037 !important;
147         border-radius: 25px !important;
148         font-weight: 900 !important;
149         color: #3E2723 !important;
150         font-size: 15px !important;
151         line-height: 1.3 !important;
152         transition: all 0.2s ease-in-out !important;
153         box-shadow: inset 0px 5px 8px rgba(0,0,0,0.35) !important;
154     }

```

```
[ ] 155
156     .mancala-pit: hover: not(:disabled) {
157         background-color: #FFE0B2 !important;
158         border-color: #8B4513 !important;
159         transform: scale(1.05) !important;
160     }
161
162     .mancala-store, .mancala-store.lm-Widget {
163         background-color: #CFD8DC !important;
164         background-image: none !important;
165         border: 4px solid #455A64 !important;
166         border-radius: 30px !important;
167         font-size: 20px !important;
168         font-weight: 900 !important;
169         color: #1A237E !important;
170         box-shadow: inset 0px 6px 12px rgba(0,0,0,0.4) !important;
171     }
172
173     .pit-label-style, .pit-label-style .widget-label {
174         color: #3E2723 !important;
175         font-weight: 900 !important;
176         font-size: 16px !important;
177         text-shadow: 1px 1px 1px rgba(255,255,255,0.6) !important;
178         text-align: center !important;
179     }
180
181     .store-title-style {
182         color: #3E2723 !important;
183         font-weight: 900 !important;
184         font-size: 16px !important;
185         text-shadow: 1px 1px 1px rgba(255,255,255,0.6) !important;
186         margin-bottom: 6px !important;
187     }
188
189     /* Enhanced Larger Status Row Layout */
190     .status-text, .status-text .widget-label {
191         font-size: 24px !important;
192         font-weight: 900 !important;
193         color: #3E2723 !important;
194         line-height: 1.5 !important;
195     }
```

```

196
197     .status-container {
198         background-color: #FFF8E1 !important;
199         padding: 20px 30px !important;
200         border-radius: 15px !important;
201         border: 3px solid #8B4513 !important;
202         border-left: 12px solid #8B4513 !important;
203         margin-bottom: 25px !important;
204         box-shadow: 0px 6px 15px rgba(0,0,0,0.15) !important;
205         width: 100% !important;
206         display: flex !important;
207         align-items: center !important;
208     }
209 </style>
210 """
211 display(HTML(custom_css))
212
213 # Graphical User Interface (GUI)
214
215 class MancalaGUI:
216     def __init__(self, ai_heuristic=4, ai_depth=4):
217         self.game = MancalaBoard()
218         self.ai_heuristic = ai_heuristic
219         self.ai_depth = ai_depth
220         self.output = widgets.Output()
221
222         # Creating a wider wrapper layout for the status bar
223
224         self.status_label = widgets.Label(value="🎲 Game Started! Your Turn (Player 1)")
225         self.status_label.add_class("status-text")
226
227         self.status_box = widgets.HBox([self.status_label], layout=widgets.Layout(width='100%'))
228         self.status_box.add_class("status-container")
229
230         self.create_board_widgets()
231         self.update_gui()
232
233     def get_marbles_text(self, count):
234         if count == 0:
235             return "Empty"
236         elif count <= 4:

```

```

236         elif count <= 4:
237             return "●" * count + f"\n({count})"
238         else:
239             return "●" * 4 + f".. \n({count})"
240
241     def create_board_widgets(self):
242         self.p2_store_btn = widgets.Button(description="", disabled=True, layout=widgets.Layout(width='100px', height='190px'))
243         self.p2_store_btn.add_class('mancala-store')
244
245         self.p1_store_btn = widgets.Button(description="", disabled=True, layout=widgets.Layout(width='100px', height='190px'))
246         self.p1_store_btn.add_class('mancala-store')
247
248         self.p2_buttons = []
249         for idx in reversed(P2_PITS):
250             btn = widgets.Button(description="", disabled=True, layout=widgets.Layout(width='85px', height='110px', margin='5px'))
251             btn.add_class('mancala-pit')
252             self.p2_buttons.append(btn)
253
254         self.p1_buttons = []
255         for idx in P1_PITS:
256             btn = widgets.Button(description="", layout=widgets.Layout(width='85px', height='110px', margin='5px'))
257             btn.add_class('mancala-pit')
258             btn.on_click(self.get_click_handler(idx))
259             self.p1_buttons.append(btn)
260
261         p2_row = widgets.HBox(self.p2_buttons)
262         p1_row = widgets.HBox(self.p1_buttons)
263
264         p2_labels = []
265         for i in reversed(P2_PITS):
266             lbl = widgets.Label(value=f"Pit {i}", layout=widgets.Layout(width='85px', justify_content='center', margin='0 5px'))
267             lbl.add_class('pit-label-style')
268             p2_labels.append(lbl)
269
270         p1_labels = []
271         for i in P1_PITS:
272             lbl = widgets.Label(value=f"Pit {i}", layout=widgets.Layout(width='85px', justify_content='center', margin='0 5px'))
273             lbl.add_class('pit-label-style')
274             p1_labels.append(lbl)
275
276         p2_labels_hbox = widgets.HBox(p2_labels)

```

```

276         p2_labels_hbox = widgets.HBox(p2_labels)
277         p1_labels_hbox = widgets.HBox(p1_labels)
278
279         middle_pits = widgets.VBox([p2_labels_hbox, p2_row, p1_row, p1_labels_hbox], layout=widgets.Layout(align_items='center'))
280
281         ai_store_lbl = widgets.Label(value="● AI Store")
282         ai_store_lbl.add_class('store-title-style')
283
284         your_store_lbl = widgets.Label(value="● Your Store")
285         your_store_lbl.add_class('store-title-style')
286
287         self.board_layout = widgets.HBox([
288             widgets.VBox([ai_store_lbl, self.p2_store_btn], layout=widgets.Layout(align_items='center')),
289             middle_pits,
290             widgets.VBox([your_store_lbl, self.p1_store_btn], layout=widgets.Layout(align_items='center'))
291         ], layout=widgets.Layout(align_items='center', justify_content='center'))
292         self.board_layout.add_class('mancala-board')
293
294     def update_gui(self):
295         self.p2_store_btn.description = f"● x {self.game.board[13]}" if self.game.board[13] > 0 else "0"
296         self.p1_store_btn.description = f"● x {self.game.board[6]}" if self.game.board[6] > 0 else "0"
297
298         for i, idx in enumerate(reversed(P2_PITS)):
299             count = self.game.board[idx]
300             self.p2_buttons[i].description = "●" * count + f"\n({count})" if count <= 4 else "●" * 4 + f".. \n({count})" if count > 0 else "Empty"
301
302         valid_moves = self.game.get_valid_moves(1)
303         for i, idx in enumerate(P1_PITS):
304             count = self.game.board[idx]
305             self.p1_buttons[i].description = "●" * count + f"\n({count})" if count <= 4 else "●" * 4 + f".. \n({count})" if count > 0 else "Empty"
306
307         if self.game.is_game_over() or self.game.current_player != 1 or idx not in valid_moves:
308             self.p1_buttons[i].disabled = True
309         else:
310             self.p1_buttons[i].disabled = False
311
312     def get_click_handler(self, pit_idx):
313         def on_button_clicked(b):
314             if self.game.current_player == 1 and not self.game.is_game_over():
315                 self.game.make_move(pit_idx)
316                 self.update_gui()

```

```

316         self.update_gui()
317
318         if self.game.is_game_over():
319             self.check_winner()
320             return
321
322         if self.game.current_player == 1:
323             self.status_label.value = "👤 Extra Move! You earned another turn."
324         else:
325             self.status_label.value = "🤖 AI is thinking..."
326             self.ai_turn()
327     return on_button_clicked
328
329     def ai_turn(self):
330         while self.game.current_player == 2 and not self.game.is_game_over():
331             time.sleep(0.8)
332             move = minimax_alpha_beta(self.game, self.ai_depth, float('-inf'), float('+inf'), True, 2, self.ai_heuristic)
333             if move is not None:
334                 self.game.make_move(move)
335                 self.update_gui()
336                 if self.game.extra_move_earned and not self.game.is_game_over():
337                     self.status_label.value = f"🤖 AI chose Pit {move} and earned an Extra Move!"
338                 else:
339                     self.status_label.value = f"🤖 AI chose Pit {move}. Now it's your turn!"
340             else:
341                 break
342         if self.game.is_game_over():
343             self.check_winner()
344
345     def check_winner(self):
346         self.update_gui()
347         p1_score = self.game.board[P1_STORE]
348         p2_score = self.game.board[P2_STORE]
349         result_text = f"🏆 Game Over! Your Score: {p1_score} | AI Score: {p2_score}. "
350         if p1_score > p2_score: result_text += "👤 You Win!"
351         elif p2_score > p1_score: result_text += "🤖 AI Wins!"
352         else: result_text += "👤 It's a Tie!"
353         self.status_label.value = result_text
354
355     def display(self):
356         with self.output:
357             display(widgets.VBox([self.status_box, widgets.Label(value=""), self.board_layout]))
358             display(self.output)
359
360 #Start the Game
361 gui = MancalaGUI(ai_heuristic=4, ai_depth=4)
362 gui.display()

```

Output:



6. Conclusion and Challenges

Conclusion:

In this project, the Mancala game was successfully implemented using the Minimax algorithm and Alpha-Beta pruning. The intelligent AI player makes optimal moves based on four engineered heuristic evaluation functions. The system is efficient and runs seamlessly in both Human vs Computer and Computer vs Computer simulation modes. This project significantly enhanced our understanding of adversarial search, tree-pruning efficiency, and strategic heuristic design.

Challenges:

Minimax & Recursion: Visualizing how the game tree simulates multiple turns ahead recursively was difficult initially.

Alpha-Beta Pruning: Implementing the pruning logic correctly without accidentally omitting optimal game moves.

State Management: Deep-copying and tracking the board states accurately during recursive search branches to avoid simulation errors.

Heuristic Engineering: Balancing different weights for storage counts, extra turns, and captured stones to build a dominant strategy.

Interactive UI Integration: Rendering a premium wood-themed graphical interface inside the notebook that syncs perfectly with AI execution delays.